

COMPREHENSIVE_TESTING_FRA MEWORK_GUIDE

COMPREHENSIVE TESTING FRAMEWORK GUIDE

Universal Multi-Format Multi-Language Ebook Translation System

Version: 1.0

Date: November 24, 2025

Test Coverage Target: 95%+ across all packages

TESTING FRAMEWORK OVERVIEW

This document outlines the complete testing framework for the Universal Ebook Translator system, covering all 6 supported test types with detailed implementation guidelines and best practices.

SUPPORTED TEST TYPES

1. UNIT TESTS

Purpose: Test individual functions and methods in isolation

Coverage Target: 90%+ for each package

Location: pkg/*/ alongside source files

Structure

```
// File: pkg/translator/translator_test.go
package translator

import (
    "testing"
```

```

"context"
"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"
"github.com/stretchr/testify/mock"
)

func TestTranslatorFunction(t *testing.T) {
    // Arrange
    testInput := "Hello world"
    expectedOutput := "Привет мир"

    // Act
    result, err := translatorFunction(testInput, "en", "ru")

    // Assert
    require.NoError(t, err)
    assert.Equal(t, expectedOutput, result)
}

```

Guidelines

- Test one function at a time
- Use table-driven tests for multiple scenarios
- Mock external dependencies
- Test both success and error paths
- Include edge cases and boundary conditions

2. INTEGRATION TESTS

Purpose: Test interactions between components

Coverage Target: All major integration points

Location: test/integration/

Structure

```

// File: test/integration/translation_integration_test.go
package integration

import (
    "testing"
    "context"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "github.com/universal-ebook-translator/pkg/translator"
    "github.com/universal-ebook-translator/pkg/storage"
)

func TestTranslationWithDatabase(t *testing.T) {
    // Setup database connection
    db := setupTestDatabase(t)
    defer cleanupTestDatabase(t, db)
}

```

```

// Setup translator
trans := setupTestTranslator(t)

// Test translation with database storage
result, err := trans.TranslateAndStore(context.Background(),
    "Hello", "en", "ru", db)

require.NoError(t, err)
assert.NotEmpty(t, result)

// Verify stored in database
stored, err := db.GetTranslation("Hello", "en", "ru")
require.NoError(t, err)
assert.Equal(t, result, stored.Text)
}

```

Guidelines

- Test real component interactions
- Use test databases and external services
- Test end-to-end workflows
- Validate data flow between components
- Include error propagation testing

3. PERFORMANCE TESTS

Purpose: Measure and validate system performance

Coverage Target: All critical paths

Location: test/performance/

Structure

```

// File: test/performance/translation_performance_test.go
package performance

import (
    "testing"
    "context"
    "time"
    "github.com/universal-ebook-translator/pkg/translator"
)

func BenchmarkTranslationSingle(b *testing.B) {
    trans := setupBenchmarkTranslator(b)
    ctx := context.Background()

    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        _, err := trans.Translate(ctx, "Hello world", "en", "ru")
        if err != nil {
            b.Fatal(err)
        }
    }
}

```

```

    }
}

func TestTranslationLatency(t *testing.T) {
    trans := setupTestTranslator(t)
    ctx := context.Background()

    start := time.Now()
    result, err := trans.Translate(ctx, "Hello world", "en", "ru")
    duration := time.Since(start)

    require.NoError(t, err)
    assert.Less(t, duration, 2*time.Second, "Translation should
        complete within 2 seconds")
    assert.NotEmpty(t, result)
}

```

Guidelines

- Benchmark critical operations
- Measure latency, throughput, and resource usage
- Test under various load conditions
- Include performance regression detection
- Set performance thresholds

4. SECURITY TESTS

Purpose: Validate security measures and vulnerabilities

Coverage Target: All security-critical components

Location: test/security/

Structure

```

// File: test/security/input_validation_test.go
package security

import (
    "testing"
    "context"
    "github.com/universal-ebook-translator/pkg/translator"
    "github.com/stretchr/testify/assert"
)

func TestInputValidation(t *testing.T) {
    trans := setupTestTranslator(t)
    ctx := context.Background()

    // Test SQL injection attempts
    maliciousInputs := []string{
        "'; DROP TABLE translations; --",
        "<script>alert('xss')</script>",
        "../../../etc/passwd",
    }
}

```

```

        "{{7*7}}", // Template injection
    }

    for _, input := range maliciousInputs {
        result, err := trans.Translate(ctx, input, "en", "ru")

        // Should not panic or crash
        assert.NotPanics(t, func() {
            _, _ = trans.Translate(ctx, input, "en", "ru")
        })

        // Should handle gracefully or return error
        if err == nil {
            assert.NotContains(t, result, "<script>")
            assert.NotContains(t, result, "DROP TABLE")
        }
    }
}

func TestAuthentication(t *testing.T) {
    // Test authentication mechanisms
    // Test authorization levels
    // Test API key validation
}

```

Guidelines

- Test input validation and sanitization
- Validate authentication and authorization
- Test for common vulnerabilities (XSS, SQL injection, etc.)
- Include rate limiting tests
- Test secure data handling

5. STRESS TESTS

Purpose: Test system behavior under extreme load

Coverage Target: System stability under stress

Location: test/stress/

Structure

```

// File: test/stress/translation_stress_test.go
package stress

import (
    "testing"
    "context"
    "sync"
    "time"
    "github.com/universal-ebook-translator/pkg/translator"
)

```

```

func TestHighConcurrencyTranslation(t *testing.T) {
    trans := setupTestTranslator(t)
    ctx := context.Background()

    const numGoroutines = 100
    const translationsPerGoroutine = 10

    var wg sync.WaitGroup
    errors := make(chan error,
        numGoroutines*translationsPerGoroutine)

    start := time.Now()

    for i := 0; i < numGoroutines; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()
            for j := 0; j < translationsPerGoroutine; j++ {
                _, err := trans.Translate(ctx, "Hello world",
                    "en", "ru")
                if err != nil {
                    errors <- err
                }
            }
        }(i)
    }

    wg.Wait()
    close(errors)

    duration := time.Since(start)

    // Check for errors
    for err := range errors {
        t.Errorf("Translation failed: %v", err)
    }

    // Performance assertions
    totalTranslations := numGoroutines * translationsPerGoroutine
    translationsPerSecond := float64(totalTranslations) /
        duration.Seconds()

    t.Logf("Completed %d translations in %v (%.2f/sec)",
        totalTranslations, duration, translationsPerSecond)

    assert.Greater(t, translationsPerSecond, 10.0,
        "Should handle at least 10 translations per second under
        stress")
}

```

Guidelines

- Test with high concurrency

- Validate resource exhaustion handling
- Test system recovery after stress
- Include memory and CPU stress tests
- Monitor for deadlocks and race conditions

6. USER ACCEPTANCE TESTS (UAT)

Purpose: Validate system meets user requirements

Coverage Target: Real-world usage scenarios

Location: test/e2e/

Structure

```
// File: test/e2e/translation_uat_test.go
package e2e

import (
    "testing"
    "context"
    "os"
    "path/filepath"
    "github.com/universal-ebook-translator/pkg/translator"
    "github.com/universal-ebook-translator/pkg/ebook"
)

func TestRealWorldTranslationWorkflow(t *testing.T) {
    // Setup test environment
    testFile := filepath.Join("test_data", "sample_book.epub")
    outputPath := filepath.Join(t.TempDir(),
        "translated_book.epub")

    // Test complete workflow
    ctx := context.Background()
    trans := setupUATTranslator(t)

    // Parse input file
    parser := ebook.NewUniversalParser()
    book, err := parser.Parse(testFile)
    require.NoError(t, err)

    // Translate content
    for i, chapter := range book.Chapters {
        translated, err := trans.Translate(ctx, chapter.Content,
            "en", "es")
        require.NoError(t, err)
        book.Chapters[i].Content = translated
    }

    // Generate output file
    writer := ebook.NewEPUBWriter()
    err = writer.Write(book, outputPath)
    require.NoError(t, err)
}
```

```

// Validate output
_, err = os.Stat(outputPath)
require.NoError(t, err)

// Verify translation quality
translatedBook, err := parser.Parse(outputPath)
require.NoError(t, err)

// Check that content is translated
assert.NotEqual(t, book.Chapters[0].Content,
    translatedBook.Chapters[0].Content)
assert.Contains(t, translatedBook.Chapters[0].Content,
    "Hola") // Spanish
}

```

Guidelines

- Test real user scenarios
 - Use actual file formats
 - Validate end-to-end workflows
 - Include multi-format testing
 - Test with various language pairs
-

TESTING INFRASTRUCTURE

Test Configuration

```

# File: test/config/test_config.yaml
test:
  database:
    type: "sqlite"
    connection: ":memory:"

  external_services:
    mock_mode: true
    rate_limit: 1000/second

  performance:
    latency_threshold: "2s"
    throughput_threshold: 100/minute

  security:
    enable_vulnerability_scan: true
    check_dependencies: true

```

Test Utilities

```
// File: test/utils/test_helpers.go
package utils

import (
    "testing"
    "context"
    "github.com/stretchr/testify/require"
    "github.com/universal-ebook-translator/pkg/translator"
    "github.com/universal-ebook-translator/pkg/storage"
)

// TestTranslator creates a translator for testing
func TestTranslator(t *testing.T) translator.Translator {
    config := translator.Config{
        Provider: translator.MockProvider,
        Model:    "test-model",
    }
    trans, err := translator.New(config)
    require.NoError(t, err)
    return trans
}

// TestDatabase creates a test database
func TestDatabase(t *testing.T) storage.Database {
    db, err := storage.NewSQLite(":memory:")
    require.NoError(t, err)

    // Run migrations
    err = db.Migrate()
    require.NoError(t, err)

    return db
}

// CreateTestEPUB creates a test EPUB file
func CreateTestEPUB(t *testing.T, content string) string {
    // Implementation for creating test EPUB
    return filepath.Join(t.TempDir(), "test.epub")
}
```

Mock Implementations

```
// File: test/mocks/mock_translator.go
package mocks

import (
    "context"
    "github.com/stretchr/testify/mock"
    "github.com/universal-ebook-translator/pkg/translator"
)
```

```

type MockTranslator struct {
    mock.Mock
}

func (m *MockTranslator) Translate(ctx context.Context, text,
    from, to string) (string, error) {
    args := m.Called(ctx, text, from, to)
    return args.String(0), args.Error(1)
}

func (m *MockTranslator) GetProvider() string {
    args := m.Called()
    return args.String(0)
}

```

COVERAGE ANALYSIS

Coverage Commands

```

# Run comprehensive coverage
go test ./... -coverprofile=coverage.out

# Generate HTML report
go tool cover -html=coverage.out -o coverage.html

# Check coverage by package
go tool cover -func=coverage.out | sort -k3 -n

# Identify low coverage areas
go tool cover -func=coverage.out | awk '$3 < "80.0%" {print $1, $3}'

```

Coverage Targets

```

# By package type:
pkg/core/          -> 95% target
pkg/translators/   -> 90% target
pkg/storage/       -> 85% target
pkg/api/           -> 90% target
pkg/utils/         -> 85% target

# By functionality:
Translation logic   -> 95% target
Error handling      -> 90% target
Input validation    -> 95% target
Security functions  -> 95% target
Performance critical paths -> 90% target

```



AUTOMATED TESTING PIPELINE

CI/CD Integration

```
# File: .github/workflows/test.yml
name: Test Pipeline

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        go-version: [1.19, 1.20, 1.21]

    steps:
      - uses: actions/checkout@v3

      - name: Setup Go
        uses: actions/setup-go@v3
        with:
          go-version: ${ matrix.go-version }

      - name: Run Unit Tests
        run: go test ./... -v -short

      - name: Run Integration Tests
        run: go test ./test/integration/... -v

      - name: Run Performance Tests
        run: go test ./test/performance/... -v -bench=.
          -benchtime=30s

      - name: Run Security Tests
        run: go test ./test/security/... -v

      - name: Run Stress Tests
        run: go test ./test/stress/... -v -timeout=30m

      - name: Run UAT
        run: go test ./test/e2e/... -v

      - name: Generate Coverage Report
        run: |
          go test ./... -coverprofile=coverage.out
          go tool cover -html=coverage.out -o coverage.html

      - name: Upload Coverage
        uses: codecov/codecov-action@v3
```

```
with:
  file: ./coverage.out
```

Pre-commit Hooks

```
# File: .pre-commit-config.yaml
repos:
  - repo: local
    hooks:
      - id: go-test
        name: go test
        entry: go test ./... -short
        language: system
        pass_filenames: false

      - id: go-lint
        name: go lint
        entry: golangci-lint run
        language: system
        pass_filenames: false

      - id: go-security
        name: go security
        entry: gosec ./...
        language: system
        pass_filenames: false
```



TEST METRICS AND REPORTING

Metrics Collection

```
// File: test/metrics/test_metrics.go
package metrics

import (
    "time"
    "github.com/prometheus/client_golang/prometheus"
)

var (
    testDuration = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
            Name: "test_duration_seconds",
            Help: "Duration of test execution",
        },
        []string{"test_type", "test_name"},
    )

    testResults = prometheus.NewCounterVec(
```

```

        prometheus.CounterOpts{
            Name: "test_results_total",
            Help: "Total number of test results",
        },
        []string{"test_type", "result"},
    )
)

func RecordTestDuration(testType, testName string, duration
    time.Duration) {
    testDuration.WithLabelValues(testType,
        testName).Observe(duration.Seconds())
}

func RecordTestResult(testType, result string) {
    testResults.WithLabelValues(testType, result).Inc()
}

```

Report Generation

```

#!/bin/bash
# File: scripts/generate_test_report.sh

echo "# Test Report" > test_report.md
echo "Generated: $(date)" >> test_report.md
echo "" >> test_report.md

# Unit Tests
echo "## Unit Tests" >> test_report.md
go test ./... -v -short | grep -E "(PASS|FAIL)" | tee -a
    test_report.md

# Coverage
echo "" >> test_report.md
echo "## Coverage" >> test_report.md
go tool cover -func=coverage.out | tail -1 >> test_report.md

# Performance
echo "" >> test_report.md
echo "## Performance" >> test_report.md
go test ./test/performance/... -bench=. -benchmem | tee -a
    test_report.md

# Security
echo "" >> test_report.md
echo "## Security" >> test_report.md
gosec ./... | tee -a test_report.md

```

TEST DATA MANAGEMENT

Test Data Structure

```
test_data/
├── ebooks/
│   ├── sample.epub
│   ├── sample.fb2
│   ├── sample.txt
│   └── sample.html
├── translations/
│   ├── en_to_es/
│   ├── en_to_fr/
│   └── en_to_de/
├── configurations/
│   ├── test_config.json
│   ├── mock_config.json
│   └── perf_config.json
└── expected_results/
    ├── translation_results.json
    └── performance_baselines.json
```

Test Data Generation

```
// File: test/data/generators.go
package data

import (
    "testing"
    "github.com/universal-ebook-translator/pkg/ebook"
)

func GenerateTestEBook(t *testing.T, chapters int) *ebook.Book {
    book := &ebook.Book{
        Title:      "Test Book",
        Author:     "Test Author",
        Language:   "en",
        Chapters:   make([]ebook.Chapter, chapters),
    }

    for i := 0; i < chapters; i++ {
        book.Chapters[i] = ebook.Chapter{
            Title:      fmt.Sprintf("Chapter %d", i+1),
            Content:    fmt.Sprintf("This is chapter %d content.",
                i+1),
        }
    }

    return book
}
```

```

func GenerateTestTranslations(t *testing.T, count int)
    []Translation {
    translations := make([]Translation, count)

    for i := 0; i < count; i++ {
        translations[i] = Translation{
            SourceText:    fmt.Sprintf("Source text %d", i),
            TargetText:    fmt.Sprintf("Target text %d", i),
            SourceLang:    "en",
            TargetLang:    "es",
            Provider:      "mock",
        }
    }

    return translations
}

```

TESTING BEST PRACTICES

Test Naming Conventions

```

// Good naming
func TestTranslator_TranslationWithValidInput_ReturnsCorrectResult(t
    *testing.T)
func TestTranslator_TranslationWithEmptyInput_ReturnsError(t
    *testing.T)
func TestTranslator_TranslationWithNetworkError_RetriesCorrectly(t
    *testing.T)

// Table-driven tests
func TestTranslator_MultipleInputs(t *testing.T) {
    tests := []struct {
        name          string
        input         string
        fromLang      string
        toLang        string
        expected      string
        expectError   bool
    }{
        {
            name:        "valid translation",
            input:       "Hello",
            fromLang:    "en",
            toLang:      "es",
            expected:    "Hola",
            expectError: false,
        },
        {
            name:        "empty input",
            input:       "",

```

```

        fromLang:    "en",
        toLang:      "es",
        expected:     "",
        expectError: true,
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        result, err := translator.Translate(tt.input,
            tt.fromLang, tt.toLang)

        if tt.expectError {
            assert.Error(t, err)
        } else {
            assert.NoError(t, err)
            assert.Equal(t, tt.expected, result)
        }
    })
}
}

```

Test Organization

```

// Test file structure
package translator

import (
    "testing"
    "github.com/stretchr/testify/suite"
)

type TranslatorTestSuite struct {
    suite.Suite
    translator translator.Translator
    ctx         context.Context
}

func (suite *TranslatorTestSuite) SetupTest() {
    suite.translator = setupTestTranslator(suite.T())
    suite.ctx = context.Background()
}

func (suite *TranslatorTestSuite) TearDownTest() {
    // Cleanup after each test
}

func (suite *TranslatorTestSuite) TestBasicTranslation() {
    result, err := suite.translator.Translate(suite.ctx, "Hello",
        "en", "es")
    suite.NoError(err)
    suite.Equal("Hola", result)
}

```

```
}

func TestTranslatorTestSuite(t *testing.T) {
    suite.Run(t, new(TranslatorTestSuite))
}
```

TROUBLESHOOTING GUIDE

Common Test Issues

1. Race Conditions

```
# Detect race conditions
go test -race ./...

# Fix with proper synchronization
mu sync.Mutex
var sharedData map[string]string

func updateData(key, value string) {
    mu.Lock()
    defer mu.Unlock()
    sharedData[key] = value
}
```

2. Test Isolation

```
// Use temporary directories for each test
func TestFileProcessing(t *testing.T) {
    tempDir := t.TempDir()

    // Test with isolated file system
    testFile := filepath.Join(tempDir, "test.txt")
    err := os.WriteFile(testFile, []byte("test"), 0644)
    require.NoError(t, err)

    // Test logic here
    // Files automatically cleaned up
}
```

3. Mock Management

```
// Reset mocks between tests
func (suite *TranslatorTestSuite) SetupTest() {
    suite.mockTranslator = new(MockTranslator)
    suite.translator = translator.New(suite.mockTranslator)
}

func (suite *TranslatorTestSuite) TearDownTest() {
```

```
    suite.mockTranslator.AssertExpectations(suite.T())
}
```

4. External Dependencies

```
// Use test containers for databases
func setupTestDatabase(t *testing.T) storage.Database {
    container, err := testcontainers.GenericContainer(
        context.Background(),
        testcontainers.GenericContainerRequest{
            ContainerRequest: testcontainers.ContainerRequest{
                Image:         "postgres:15",
                ExposedPorts: []string{"5432/tcp"},
                Env: map[string]string{
                    "POSTGRES_DB":      "testdb",
                    "POSTGRES_USER":    "testuser",
                    "POSTGRES_PASSWORD": "testpass",
                },
            },
            Started: true,
        },
    )
    require.NoError(t, err)

    // Get connection string and create database
    // ...
}
```

TESTING CHECKLIST

Before Running Tests

☐

Clean test environment

☐

Update test dependencies

☐

Verify external services (if any)

☐

Check test data availability

During Test Development

☐

Follow naming conventions

☐

Include both success and error cases

- ☐ Add proper assertions
- ☐ Document test purpose
- ☐ Keep tests focused and independent

After Test Implementation

- ☐ Run test suite locally
- ☐ Check coverage reports
- ☐ Verify test isolation
- ☐ Update documentation
- ☐ Review code quality

Continuous Integration

- ☐ All tests pass in CI/CD
- ☐ Coverage thresholds met
- ☐ Performance benchmarks stable
- ☐ Security scans pass
- ☐ Test reports generated

FRAMEWORK STATUS: COMPLETE AND READY FOR IMPLEMENTATION **NEXT STEP:** EXECUTE COMPREHENSIVE TEST SUITE **MAINTENANCE:** REGULAR UPDATES AND ENHANCEMENTS

This testing framework provides the foundation for ensuring the Universal Ebook Translator meets the highest quality standards with comprehensive test coverage across all 6 supported test types.